

UNIDAD 1 PARTE 1– RESOLUCION PROB

Un **estado** describe los elementos del problema en un instante dado. Deben incluirse los elementos que cambian en el tiempo y que no pueden determinarse a partir de otros elementos.

Acciones: modifica el estado actual

Formular: Proceso de expresar a un problema como un problema de búsqueda, debiendo elegir una **representación concreta** para sus **estados** y sus **acciones**

Abstraer: Proceso de remover de una representación detalles **irrelevantes**.

Un problema de búsqueda se define por:

1. **Estado inicial:** Estado del problema en el instante inicial.
2. **Acciones:** Dado un estado S , la función **ACCIONES(S)** retorna el conjunto de acciones que se pueden ejecutar en S .
3. **Modelo de transiciones:** Describe el resultado de aplicar las acciones a los estados. Dada un estado S y una acción $A \in \text{ACCIONES}(S)$, la función **RESULTADO(S, A)** retorna el estado que resulta de aplicar A en S .
Si $\text{RESULTADO}(S, A) = S'$, entonces S' es un **sucesor** de S .
4. **Test objetivo** Dado un estado S , el predicado **TEST-OBJETIVO(S)** determina si S es un estado objetivo
5. **Costo de camino** A cada **camino** del espacio de estados se le puede asignar un costo numérico (no negativo), denominado **costo de camino**. Dado un camino P , la función **COSTO-CAMINO(P)** retorna el costo de camino de P . El costo de un camino es igual a la suma de los costos individuales de cada acción del camino.
Dado un estado S y una acción $A \in \text{ACCIONES}(S)$, la función **COSTO-INDIVIDUAL(S, A)** retorna el costo individual de aplicar A en S

El estado inicial, las acciones y el modelo de transiciones definen implícitamente el **espacio de estados** del problema. Cuando es finito, se puede representar con un **grafo dirigido** (o **digrafo**) donde los **nodos** son estados y los **arcos** son acciones. El **tamaño** del espacio de estados es el total de estados/nodos.

Un **camino** en el espacio de estados es una secuencia de estados conectados mediante una secuencia de acciones.

Estado objetivo: Es un estado al que se busca llegar. **Pueden existir múltiples estados objetivos.**

Una **solución** es un **camino** en el espacio de estados desde el **estado inicial** a un **objetivo**. Una **solución óptima** tiene el **menor costo de camino** entre todas las soluciones.

UNIDAD 1 PARTE 2 – RESOLUCION PROB

- Un **algoritmo de búsqueda** recibe un problema de búsqueda cualquiera y trata de retornar una **solución**, idealmente **óptima**.
- Problemas reales tienen **espacios de estados** grandes y su **grafo de estados** no se puede almacenar completamente en memoria.

ALCANZADOS Conjunto con los **estados** de los nodos generados (expandidos o no)

FRONTERA Conjunto con los **nodos** generados y no expandidos. **Candidatos a expandir.**

BUSQUEDA DE GRAFO

1. **Inicialización.** Generar la raíz n_0 con el estado inicial s_0 . Poner en ALCANZADOS a s_0 y en FRONTERA a n_0 .
 2. **Iteración.** Mientras FRONTERA no esté vacía:
 3. **Selección.** Sacar un nodo n de FRONTERA.
 4. **Test objetivo.** Retornar si n tiene un estado objetivo.
 5. **Expansión.** Expandir n y actualizar FRONTERA Y ALCANZADOS con sus vecinos no alcanzados.
- el test objetivo lo aplicamos luego de sacar un nodo de la frontera (y no al agregarlo), tenemos que seguir

ARBOL DE BUSQUEDA

- Cada **nodo** contiene un estado. En particular, la **raíz** contiene el estado inicial.
- De cada nodo (**padre**) y por cada acción, hay una **rama** que lo conecta con un nodo con el estado sucesor (**hijo**).
- Una **hoja** es un nodo sin hijos (estados sin sucesores).
- El **nivel** o **profundidad** de un nodo es el largo del camino que conecta ese nodo con la raíz.

NODO Un nodo n se puede representar con una estructura con 4 (o 5) atributos:

- **n .estado:** estado almacenado.
- **n .padre:** nodo padre.
- **n .acción:** acción aplicada en el padre.
- **n .costo:** costo de camino de la raíz a n .
- **n .profundidad** (opcional): profundidad de n .

¿Por qué se guarda referencia al padre y no al hijo? **Ventaja.** Podemos recuperar fácilmente una **solución**. Una vez encontrado un nodo objetivo, retrocedemos por los padres hasta la raíz

Algoritmo general de búsqueda de grafo

```
1 function GRAPH-SEARCH(problema) return solución o no-solución
2    $n_0 \leftarrow \text{NODO}(\text{problema.estado-inicial}, \text{None}, \text{None}, 0)$ 
3   ALCANZADOS  $\leftarrow \{n_0.\text{estado}\}$ 
4   FRONTERA  $\leftarrow \{n_0\}$ 
5   do
6     if FRONTERA.empty() then return no-solución
7      $n \leftarrow \text{FRONTERA.pop}()$ 
8     if problema.test-objetivo( $n.\text{estado}$ ) then return solución
9     for all a in problema.acciones( $n.\text{estado}$ ) do
10        $s' \leftarrow \text{problema.resultado}(n.\text{estado}, a)$ 
11       if  $s'$  not in ALCANZADOS then
12         ALCANZADOS.insert( $s'$ )
13          $n' \leftarrow \text{NODO}(s', n, a, n.\text{costo} + \text{problema.c}(n.\text{estado}, a))$ 
14         FRONTERA.push( $n'$ )
```

Notar que siempre que se **agrega** un **nodo** a **FRONTERA**, su estado **también** se agrega a **ALCANZADOS**.

el consumo de memoria de GRAPH-SEARCH depende principalmente de: **El máximo número de estados en ALCANZADOS**. **ALCANZADOS** podría contener tantos estados como estados alcanzables tenga el problema.

BUSQUEDA DE ARBOL

La **búsqueda de árbol** es similar a la búsqueda de grafo, salvo que **no se mantiene en memoria un conjunto con los estados alcanzados previamente**. El árbol de búsqueda que se construye podría contener nodos con estados **repetidos**

Búsqueda de árbol

1. **Inicialización**. Generar la raíz n_0 con el estado inicial s_0 . ~~Poner en ALCANZADOS a s_0 y en FRONTERA a n_0 .~~
2. **Iteración**. Mientras FRONTERA no esté vacía:
3. **Selección**. Sacar un nodo n de FRONTERA.
4. **Test objetivo**. Retornar si n es objetivo.
5. **Expansión**. Expandir n y actualizar FRONTERA ~~Y ALCANZADOS~~ con sus vecinos ~~no alcanzados~~.



Algoritmo general de búsqueda de árbol

```
1 function TREE-SEARCH(problema) return solución o no-solución
2    $n_0 \leftarrow \text{NODO}(\text{problema.estado-inicial}, \text{None}, \text{None}, 0)$ 
3   FRONTERA  $\leftarrow \{n_0\}$ 
4   do
5     if FRONTERA.empty() then return no-solución
6      $n \leftarrow \text{FRONTERA.pop}()$ 
7     if problema.test-objetivo( $n$ .estado) then return solución
8     for all  $a$  in problema.acciones( $n$ .estado) do
9        $s' \leftarrow \text{problema.resultado}(n.estado, a)$ 
10       $n' \leftarrow \text{NODO}(s', n, a, n.costo + \text{problema.c}(n.estado, a))$ 
11      FRONTERA.push( $n'$ )
```

Un **camino cíclico** en el grafo de estados es un camino que pasa al menos dos veces por un mismo estado. En presencia de caminos cíclicos, **el árbol de búsqueda podría ser infinito y TREE-SEARCH podría no terminar**

Detección de caminos cíclicos

- Antes de agregar un nodo n a FRONTERA, controlamos **si el camino desde n a la raíz tiene estados repetidos**.
 - Si repite, **podamos** a n .
 - Sino, **agregamos** n a FRONTERA.
- Dado que cada nodo almacena a su padre, este control es fácil de agregar y no es demasiado costoso.
- La detección de ciclos garantiza terminación en espacios de estados finitos.

Algoritmo general de búsqueda de árbol con detección de ciclos

```
1 function TREE-SEARCH(problema) return solución o no-solución
2    $n_0 \leftarrow \text{NODO}(\text{problema.estado-inicial}, \text{None}, \text{None}, 0)$ 
3   FRONTERA  $\leftarrow \{n_0\}$ 
4   do
5     if FRONTERA.empty() then return no-solución
6      $n \leftarrow \text{FRONTERA.pop}()$ 
7     if problema.test-objetivo( $n.\text{estado}$ ) then return solución
8     for all  $a$  in problema.acciones( $n.\text{estado}$ ) do
9        $s' \leftarrow \text{problema.resultado}(n.\text{estado}, a)$ 
10      if not hay_ciclo( $s', n$ ) then
11         $n' \leftarrow \text{NODO}(s', n, a, n.\text{costo} + \text{problema.c}(n.\text{estado}, a))$ 
12        FRONTERA.push( $n'$ )
```

hay_ciclo(s', n): función que toma un estado s' y un nodo n y determina si s' se repite en el camino desde n a la raíz.

¿Será ésta la solución **definitiva** a la repetición de estados? 😞

Caminos redundantes

En el grafo de estados, dos caminos **distintos** con mismo origen y mismo destino se llaman **caminos redundantes**.

La existencia de caminos redundantes puede generar que el árbol de búsqueda construido por TREE-SEARCH tenga nodos con estados repetidos, incluso si se detectan ciclos.

Un estado podría repetirse en el árbol de búsqueda tantas veces como caminos redundantes haya desde el estado inicial hasta a él.

El número de estados a distancia a lo sumo d de s_0 responde a la fórmula: $2d^2 + 2d + 1$

Para evitar cualquier camino redundante hay que recordar los estados **alcanzados** en el pasado, es decir, volver a la búsqueda de grafo

UNIDAD 2 PARTE 1 – NO INFORMADA

1. **No-informadas.** Toman decisiones basadas únicamente en la definición del problema de búsqueda.
2. **Informadas.** Cuentan con información adicional para saber cuándo un estado no-objetivo es más prometedor que otro.

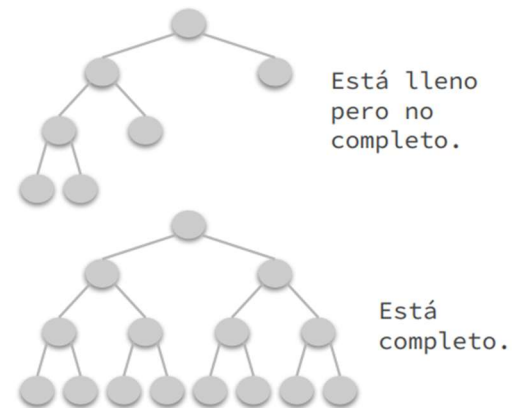
Árbol k-ario

Definición 1. Dado $k \in \mathbb{N}$, un árbol es **k-ario** si cada nodo tiene a lo sumo k hijos. Un **árbol binario** es un caso particular de árbol k-ario para $k = 2$.

Definición 2. El **factor de ramificación b** es el máximo número de hijos de cualquier nodo, es decir, el mínimo $k \in \mathbb{N}$ tal que el árbol es k-ario

Definición 3. Un árbol k-ario está **lleno** si todos los nodos tienen 0 o k hijos.

Definición 4. Un árbol k-ario está **completo** si está **lleno** y todas las hojas están en el **mismo nivel**

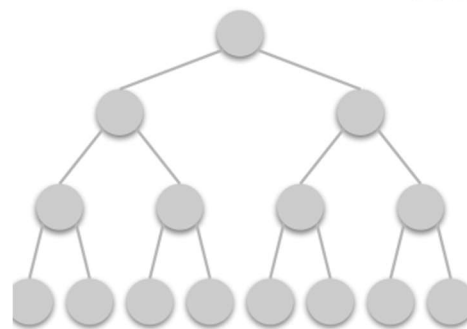


Número de nodos

Dado un árbol completo con **factor de ramificación b**, el número de nodos en el **nivel d** es b^d

Hasta el nivel d es $(b^{d+1} - 1) / (b - 1)$

Ejemplo árbol con $b=2$



Nivel d	Nodos en d	Nodos en d o menos
0	$2^0=1$	$(2^1-1)/1=1$
1	$2^1=2$	$(2^2-1)/1=3$
2	$2^2=4$	$(2^3-1)/1=7$
3	$2^3=8$	$(2^4-1)/1=15$

Búsqueda primero en anchura o Breadth-First Search (BFS)

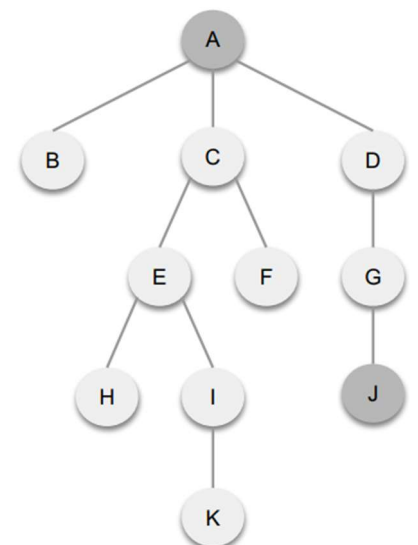
Primero se expande la raíz, luego los hijos de la raíz, luego los hijos de los hijos de la raíz, y así hasta encontrar un estado objetivo.

En la estrategia BFS, es **conveniente** ejecutar el test objetivo **ANTES de agregar un nodo a la frontera** (En el mejor de los casos, **evitamos generar un nivel completo del árbol**)

1. A, B, C, D, E, F, G. ✓ Los nodos se expanden por niveles. Primero A con nivel 0. Luego B, C y D con nivel 1. Luego E, F y G con nivel 2. Tras expandir a G, se genera a J y el algoritmo se detiene.

2. A, D, B, C, G. ✓ Los nodos también se expanden por niveles. Primero A con nivel 0. Luego D, B y C con nivel 1 (no se expanden de izquierda a derecha pero esto no importa). Luego G en el nivel 2. Tras expandir a G, se genera a J y el algoritmo se detiene (el nivel 2 no fue expandido por completo, pero esto tampoco importa).

3. A, D, B, C, E, F, I, G. ✗ El nodo I con nivel 3 se expande antes que el nodo G con nivel 2



Implementación de BFS

- ¿Qué nodo se elige de la frontera? El de **menor** profundidad.
- ¿Cómo lo logramos? Si los nuevos nodos generados (que están un nivel más abajo que sus padres) se agregan al **final** de la frontera, entonces en la parte de **adelante** de la frontera siempre se encuentran los de menor nivel.

-> **TAD COLA FIFO**: el primero que entra es el primero que sale



Algoritmo BFS de árbol

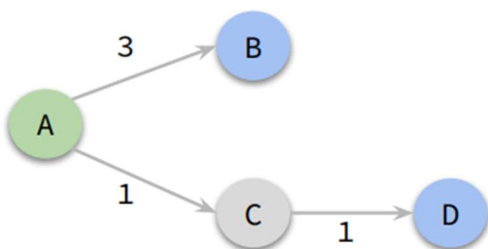
```
1 function TREE-BFS(problema) return solución o fallo
2   n0 ← NODO(problema.estado-inicial, None, None, 0)
3   if (problema.test-objetivo(n0.estado)) then return solución(n0)
4   frontera ← Cola()
5   frontera.encolar(n0)
6   do
7     if frontera.vacía() then return fallo
8     n ← frontera.desencolar()
9     forall a in problema.acciones(n.estado) do
10      s' ← problema.resultado(n.estado, a)
11      n' ← NODO(s', n, a, n.costo + problema.costo-individual(n.estado,a))
12      if problema.test-objetivo(s') then return solución(n')
13      frontera.encolar(n')
```

solución(n): retorna la solución escalando por los padres desde el nodo n hasta la raíz.

El test objetivo (líneas 3 y 12) se aplica **antes** de agregar un nodo a frontera.

Suponer que el árbol de búsqueda tiene un factor de ramificación **b** y que la menor profundidad de una solución es **d**

- Si existen soluciones, ¿encuentra al menos una? **Siempre* encuentra una solución, aunque no se detecten caminos cíclicos**. De hecho encuentra una de profundidad mínima (**d**). Salvo que el factor de ramificación sea **infinito**.
- Si existen soluciones, ¿encuentra una óptima? ☒ **Para costos individuales unitarios (= 1), siempre encuentra una solución óptima**. En estos casos, la **profundidad** de un nodo es exactamente su **costo de camino**. Para otros costos individuales, puede no ser óptimo.



En este caso, TREE-BFS encuentra la solución A → B de nivel 1, la cual no es óptima porque su costo es 3 y existe la solución A → C → D en el nivel 2 con costo 2.

- **Consumo de tiempo de TREE-BFS** En el peor caso, se generan todos los nodos hasta el nivel **d**
$$(b^{d+1} - 1)/(b - 1) \leq b^{d+1}$$
- **Consumo de memoria de TREE-BFS** El máximo número de nodos en la frontera es a lo sumo el número de nodos en el nivel **d** b^d

Sin embargo, **sus ancestros no pueden liberarse de la memoria, ya que son necesarios para reconstruir la solución**. En el peor caso, se deben almacenar en memoria todos los nodos hasta el nivel d $(b^{d+1} - 1)/(b - 1) \leq b^{d+1}$

Performance de TREE-BFS

- **Completitud.** ✓
- **Optimalidad.** ✓ Cuando las acciones tienen costo individual unitario. ✗ En otros casos.
- **Tiempo.** Se generan a lo sumo b^{d+1} nodos.
- **Memoria.** Se mantienen a lo sumo b^{d+1} nodos en memoria.

Búsqueda primero en profundidad o Depth-First Search (DFS)

Comenzando por la raíz, se expande uno de los hijos del último nodo procesado. La búsqueda desciende rápidamente a una hoja.

Para continuar, se retrocede a su ancestro más profundo con hijos aún no expandidos y se elige uno de ellos.

En DFS **el test objetivo se llama antes de agregar un nuevo nodo a la frontera** (al igual que en BFS). Con respecto a llamarlo después de sacar de la frontera, se logra:

✓ **Menor consumo de tiempo** (aunque el ahorro es difícil de medir)

🔥 Para costos individuales no unitarios, nada garantiza que la solución encontrada antes sea mejor/igual/peor.

1. **A, C, G.**

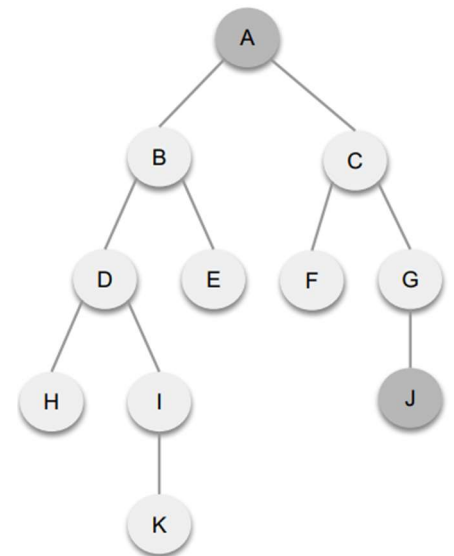
✓ El orden de expansión sigue un camino descendente en el árbol. El algoritmo se detiene tras expandir a G y generar a J.

2. **A, C, F, G.**

✓ El orden de expansión sigue un camino descendente en el árbol hasta F. Dado que F es una hoja, la búsqueda retrocede a su padre C y continúa expandiendo su otro hijo G. El algoritmo se detiene tras expandir a G y generar a J.

3. **A, B, E, C, G.**

✗ El orden de expansión sigue un camino descendente en el árbol hasta E. Pero dado que E es una hoja, la búsqueda tendría que haber retrocedido hasta su padre B y continuado expandiendo su otro hijo D.



Implementación de DFS

- ¿Qué nodo se elige de la frontera? El de **mayor** profundidad.
- ¿Cómo lo logramos? Si los nuevos nodos generados (que están un nivel más abajo que sus padres) se agregan al **final** de la frontera, entonces en la parte de **atrás** de la frontera siempre se encuentran los de mayor nivel -> **TAD Pila LIFO**: el último que entra es el primero que sale.



Algoritmo DFS de árbol

```

1 function TREE-DFS(problema) return solución o fallo
2   n0 ← NODO(problema.estado-inicial, None, None, 0)
3   if (problema.test-objetivo(n0.estado)) then return solución(n0)
4   frontera ← Pila()
5   frontera.apilar(n0)
6   do
7     if frontera.vacia() then return fallo
8     n ← frontera.desapilar()
9     forall a in problema.acciones(n.estado) do
10      s' ← problema.resultado(n.estado, a)
11      n' ← NODO(s', n, a, n.costo + problema.costo-individual(n.estado, a))
12      if not hay_ciclo(s', n) then
13        if problema.test-objetivo(s') then return solución(n')
14        frontera.apilar(n')

```

Suponer que el árbol de búsqueda tiene un factor de ramificación b y que tiene m niveles (**altura** del árbol) $m \geq d$, siendo d la menor profundidad con un nodo objetivo. Suponemos que árbol es finito

- Si existen soluciones, ¿encuentra al menos una? Si **se detectan caminos cíclicos**, entonces el árbol de búsqueda es **finito*** y DFS siempre encuentra una solución.
- Si existen soluciones, ¿encuentra una óptima? No, ya lo sabemos del ejemplo anterior
- **Consumo de tiempo de TREE-DFS** En el peor caso, se generan todos los nodos hasta el nivel d

$$(b^{m+1} - 1)/(b - 1) \leq b^{m+1}$$

- **Consumo de memoria de TREE-DFS** Es necesario mantener en memoria los **nodos del camino desde n a la raíz** (a lo sumo m nodos) y a la frontera, que contiene a **sus hermanos no expandidos** (a lo sumo b nodos por cada nodo del camino). El máximo número de nodos en memoria está acotado por: $b \cdot m$

PERFORMANCE de TREE-DFS

- **Complejidad.** ✗ Si el espacio de estados es infinito o es finito pero no se detectan ciclos.

✓ Si el espacio de estados es finito y se detectan ciclos.

- **Optimalidad.** ✗
- **Tiempo.** Se generan a lo sumo b^{m+1} nodos. Si se detectan ciclos, el consumo de tiempo es $m \cdot b^{m+1}$
- **Memoria.** Se mantienen a lo sumo $b \cdot m$ nodos en memoria. **Es lineal**

El requerimiento de memoria de DFS puede llegar a ser varios órdenes de magnitud menor al de BFS, pero se pierde la garantía de optimalidad.

La adaptación de BFS de árbol a grafo. • **ALCANZADOS:** conjunto de estados alcanzados.

- Al expandir un nodo, se descartan (**podan**) los hijos con estados ya alcanzados. Para los restantes, se marcan sus estados como alcanzados y se encolan a la frontera.



Algoritmo BFS de grafo

```
1 function GRAPH-BFS(problema) return solución o fallo
2   n0 ← NODO(problema.estado-inicial, None, None, 0)
3   if (problema.test-objetivo(n0.estado)) then return solución(n0)
4   frontera ← Cola()
5   frontera.encolar(n0)
6   alcanzados ← {n0.estado}
7   do
8     if frontera.vacia() then return fallo
9     n ← frontera.desencolar()
10    forall a in problema.acciones(n.estado) do
11      s' ← problema.resultado(n.estado, a)
12      if s' is not in alcanzados then
13        n' ← NODO(s', n, a, n.costo + problema.costo-individual(n.estado, a))
14        if problema.test-objetivo(s') then return solución(n')
15      alcanzados.insertar(s')
16    frontera.encolar(n')
```

- ¿Cómo justificamos que la poda es **segura**? Si la expansión genera un nodo con un estado alcanzado previamente, entonces ese estado se **alcanzó** por primera vez en un **nivel** menor o igual. Luego, **el nodo podado conduce a un camino redundante de largo mayor o igual**.
- Por lo tanto, **es completo y óptimo para costos individuales unitarios**
- ¿Cuál es el consumo de **tiempo** y **memoria** de GRAPH-BFS comparado con TREE-BFS?

Si hay muchos caminos redundantes, el ahorro de GRAPH-BFS puede ser significativo. Aunque en el peor caso, se generan **tantos nodos como estados alcanzables**.

Si no hay caminos redundantes, consumen mismo tiempo y GRAPH-BFS más memoria por los estados alcanzados, aunque no es significativo (recordar que TREE-BFS también recuerda la frontera y sus ancestros)



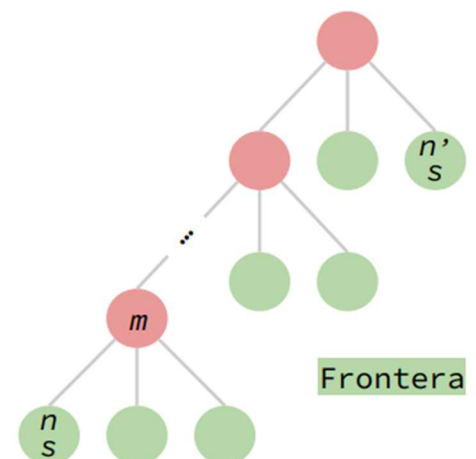
Algoritmo DFS de grafo

Suponer que al expandir un nodo **m**, se genera un hijo **n** con un estado **s** tal que en la frontera hay un nodo **n'** con el mismo estado **s** en algún nivel anterior (generado previamente).

Manteniendo un conjunto de estados **alcanzados**, el nodo **n** sería descartado (pues **s** ya fue alcanzado al generar a **n'**), cuando lo correcto sería descartar a **n'** pues tiene **menor profundidad**.

En lugar de mantener un conjunto de estados alcanzados, se mantiene un conjunto de estados **expandidos** (provenientes de nodos ya sacados de la frontera)

Ahora, en la frontera pueden aparecer **nodos con estado repetidos**, pero solo se permite expandir el de mayor profundidad ⇒ **Se agrega un control que evita expandir un nodo con un estado ya expandido**.





Algoritmo DFS de grafo

```

1 function GRAPH-DFS(problema) return solución o fallo
2   n0 ← NODO(problema.estado-inicial, None, None, 0)
3   if problema.test-objetivo(n0.estado) then return solución(n0)
4   frontera ← Pila()
5   frontera.apilar(n0)
6   expandidos ← {}
7   do
8     if frontera.vacia() then return fallo
9     n ← frontera.desapilar()
10    if n.estado is in expandidos then continue
11    expandidos.insertar(n.estado)
12    forall a in problema.acciones(n.estado) do
13      s' ← problema.resultado(n.estado, a)
14      if s' is not in expandidos then
15        n' ← NODO(s', n, a, n.costo + problema.costo-individual(n.estado, a))
16        if problema.test-objetivo(s') then return solución(n')
17        frontera.apilar(n')

```

Control que evita expandir un estado ya expandido.

Performance de GRAPH-DFS

- Al igual que TREE-BFS, es **completo** en espacios de estados finitos y puede ser **subóptimo** (devolver solución no óptima).
- El **consumo de tiempo** depende de la cantidad de caminos redundantes del problema (similar a GRAPH-BFS).
- 💡 El **consumo de memoria aumenta**. Deja de ser lineal en la altura, pues ahora se almacenan todos los estados **expandidos**, que pueden ser tantos como el número de estados alcanzables

🔴 GRAPH-BFS vs. GRAPH-DFS

En el peor caso, tienen el mismo requerimiento de tiempo y memoria, pero GRAPH-BFS es óptimo (para costos individuales unitarios).

Luego, **se prefiere a GRAPH-BFS como algoritmo de búsqueda de grafo.**

UNIDAD 2 PARTE 2 – NO INFORMADA

- La **optimalidad** de BFS depende de los costos individuales.
- La **completitud** de DFS depende de la finitud del árbol de búsqueda y no garantiza **optimalidad**.

Búsqueda de costo uniforme o Uniform-Cost Search (UCS)

UCS es una **generalización** de BFS que extiende la optimalidad para problemas con **cualquier función de costo individual**. Se expande siempre el de **menor costo de camino**.

En particular, cuando los costos individuales son unitarios, el costo de camino coincide con la profundidad y la estrategia es **similar** a BFS.

Los estados alcanzados los vamos a almacenar en un **diccionario**, donde las **claves** son los estados y los **valores** son costos de camino. 💡 Cuando UCS alcanza un estado (objetivo o no) por primera vez, **no garantiza** un camino de costo mínimo a ese estado.

👉 Cuando UCS extrae un nodo de la frontera, entonces **garantiza** un camino de costo mínimo a ese estado. Los demás nodos de la frontera tienen un costo de camino que es mayor o igual (de lo contrario hubiesen sido extraídos anteriormente) y no pueden conducir a caminos redundantes de menor costo (pues los costos individuales nunca son negativos). En la estrategia UCS, es conveniente ejecutar el test objetivo **después de sacar un nodo de la frontera**.

Implementación de UCS

- ¿Qué nodo se elige de la frontera? El de menor costo de camino.
- ¿Cómo lo logramos? Si los nuevos nodos se insertan ordenados en la frontera según su costo de camino, entonces en la parte de adelante de la frontera siempre se encuentra el nodo con mínimo costo de camino -> **TAD Cola de prioridad** Política: el de menor prioridad es el primero que sale.



Algoritmo UCS de grafo

```

1 function GRAPH-UCS(problema) return solución o no-solución
2   n0 ← NODO(problema.estado-inicial, None, None, 0)
3   frontera ← ColaPrioridad()
4   frontera.encolar(n0, n0.costo)
5   alcanzados ← {n0.estado: n0.costo}
6   do
7     if frontera.vacía() then return no-solución
8     n ← frontera.desencolar()
9     if problema.test-objetivo(n.estado) then return solución(n)
10    forall a in problema.acciones(n.estado) do
11      s' ← problema.resultado(n.estado, a)
12      c' ← n.costo + problema.costo-individual(n.estado, a)
13      if s' is not in alcanzados or c' < alcanzados[s'] then
14        n' ← NODO(s', n, a, c')
15        alcanzados[s'] ← c'
16        frontera.encolar(n', c')

```

TREE-UCS es muy similar. Hay que borrar las partes relacionadas con el diccionario de alcanzados (líneas 5, 13 y 15).

Un nodo se **descarta** únicamente cuando su estado ya había sido alcanzado con un costo de camino menor o igual.

Performance de TREE-UCS y GRAPH-UCS

- **Completitud y optimalidad.** ✅ Para costos individuales $> 0^*$

*TREE-UCS podría no terminar si hay un camino cíclico en el espacio de estados con acciones de costo individual 0.

- **Tiempo y memoria.** El consumo de tiempo y memoria de TREE-UCS sigue siendo **exponencial**, aunque el exponente depende ahora del cociente entre el costo de una solución óptima y el menor costo individual de una acción.

Búsqueda de profundización iterativa o Iterative-Deepening Search (IDS)

Se introduce un **límite de profundidad** l . En cada iteración, se aplica **DFS** sobre el **árbol de búsqueda limitado** a profundidad l .

Es decir, los nodos del árbol de búsqueda en el nivel l se tratan como si fuesen **hojas**.

Comenzando con $l = 0$, el límite se **aumenta** iterativamente en 1 hasta encontrar una solución

TREE-LIMITED-DFS puede devolver una **solución**, un **fallo** o un **seguir**, este último indica que se **alcanzó** el límite de **profundidad** sin encontrar solución y se **precisa aumentarlo**

```

1 function TREE-IDS(problema) return solución o no-solución
2   for l = 0 to ∞ do
3     resultado ← TREE-LIMITED-DFS(problema, l)
4     if resultado ≠ seguir then resultado

```



Algoritmo IDS de árbol

```

1 function TREE-LIMITED-DFS(problema, l) return solución o no-solución
2   n0 ← NODO(problema.estado-inicial, None, None, 0, profundidad=0)
3   if problema.test-objetivo(n0.estado) then return solución(n0)
4   frontera ← Pila()
5   frontera.apilar(n0)
6   retorno ← no-solución
7   do
8     if frontera.vacía() then return retorno
9     n ← frontera.desapilar()
10    if n.profundidad == l then retorno ← seguir
11    else
12      forall a in problema.acciones(n.estado) do
13        s' ← problema.resultado(n.estado, a)
14        n' ← NODO(s', n, a, n.costo+problema.costo-individual(n.estado, a),
15                  profundidad=n.profundidad+1)
16        if problema.test-objetivo(s') then return solución(n')
17        frontera.apilar(n')

```

Se precisa el atributo **profundidad** del nodo.

Si no encuentra solución retorna:
 - **no-solución** si no se generó ningún nodo de profundidad $l \rightarrow$ **no existe solución.**
 - **seguir** si se generó al menos un nodo de profundidad $l \rightarrow$ **se debe aumentar un nivel.**

Si alguno de los nodos extraídos se encuentra en el límite de profundidad, entonces se **descarta sin expandir** y se marca el retorno como **seguir**.

Performance de TREE-IDS

- Si existen soluciones, ¿encuentra al menos una?

La primera llamada a TREE-LIMITED-DFS genera el árbol hasta el nivel 0.

La segunda llamada a TREE-LIMITED-DFS genera el árbol hasta el nivel 1.

La tercera llamada a TREE-LIMITED-DFS genera el árbol hasta el nivel 2. ... y así sucesivamente hasta el nivel d .

Siempre⁺ encuentra una solución aunque no se detecten caminos cíclicos. De hecho la encuentra al llegar al límite de profundidad $l = d$. ***Si el límite de profundidad es lo suficientemente grande.**

- Si existen soluciones, ¿encuentra una óptima? **Para costos individuales unitarios (= 1), siempre encuentra una solución óptima**
- **Completitud.** ✓
- **Optimalidad.** ✓ Si las acciones tienen costo individual unitario. ✗ En otros casos.
- **Tiempo.** Se generan a lo sumo b^{d+1} nodos.
- **Memoria.** Se mantienen a lo sumo $b * d$ nodos en memoria. **Es lineal**

Combina las ventajas de TREE-BFS (completitud y optimalidad) y TREE-DFS (memoria). Es el algoritmo de búsqueda de árbol preferido en grandes espacios de búsqueda y con pocos caminos redundantes.

UNIDAD 3 PARTE 1 – INFORMADAS

- Saben si un nodo tiene un estado **más prometedor** que otro y lo expanden antes.
- Requieren cierto **conocimiento específico** del problema
- Incorporan una **función heurística** h donde, para cada nodo n , $h(n)$: **es una estimación del menor costo de camino desde el estado de n a un estado objetivo.**
- Si n es un nodo con un estado objetivo, es necesario que h verifique $h(n) = 0$.

Búsqueda avara primero el mejor o Greedy Best-First Search (GBFS)

Los nodos de la frontera se expanden según su cercanía a un nodo objetivo.

Dada una **función heurística** h , se expande siempre el nodo n de la frontera con el **menor valor heurístico**.

Observaciones:

1. Puede seguir caminos cíclicos. Es necesario detectarlos para garantizar **completitud** en espacios de estados finitos.
2. Tiende a seguir caminos en profundidad. Pero a diferencia de DFS, puede **retroceder** antes de llegar a una hoja si el camino se empieza a alejar del objetivo.
3. No tiene garantía de **optimalidad**

Implementación GBFS

- ¿Cómo elegimos de la frontera el próximo nodo a expandir? El nodo n con el menor costo $h(n)$.
- ¿Cómo lo logramos? Al igual que en UCS, usando el TAD **cola de prioridad** para la frontera pero ordenando los nodos por la función heurística.



Algoritmo GBFS de grafo

```
1 function GRAPH-GBFS(problema) return solución o fallo
2    $n_0 \leftarrow \text{NODO}(\text{problema.estado-inicial}, \text{None}, \text{None}, 0)$ 
3   frontera  $\leftarrow \text{ColaPrioridad}()$ 
4   frontera.encolar( $n_0$ ,  $\text{problema.h}(n_0)$ )
5   alcanzados  $\leftarrow \{n_0.\text{estado}: n_0.\text{costo}\}$ 
6   do
7     if frontera.vacia() then return fallo
8      $n \leftarrow \text{frontera.desencolar}()$ 
9     if problema.test-objetivo( $n.\text{estado}$ ) then return solución( $n$ )
10    forall  $a$  in problema.acciones( $n.\text{estado}$ ) do
11       $s' \leftarrow \text{problema.resultado}(n.\text{estado}, a)$ 
12       $c' \leftarrow n.\text{costo} + \text{problema.costo-individual}(n.\text{estado}, a)$ 
13      if  $s'$  is not in alcanzados or  $c' < \text{alcanzados}[s']$  then
14         $n' \leftarrow \text{Nodo}(s', n, a, c')$ 
15         $\text{alcanzados}[s'] \leftarrow c'$ 
16        frontera.encolar( $n'$ ,  $\text{problema.h}(n')$ )
```

Suponemos que el problema tiene un **método** h que implementa la heurística.

TREE-GBFS es análogo. Hay que borrar únicamente las partes relacionadas con el diccionario de alcanzados y agregar la detección de ciclos.

PERFORMANCE GBFS

- **Completitud de TREE-GBFS.** ❌ Si el espacio de estados es infinito o es finito pero no se detectan ciclos. ✅ Si el espacio de estados es finito y se detectan ciclos

- **Optimalidad.** ❌

En el peor caso, la heurística podría ser $h(n) = 0$ para todo nodo n , es decir, todos los nodos de la frontera tienen la misma prioridad y la búsqueda se vuelve no informada.

Por lo tanto podría tener el mismo **consumo de tiempo** que TREE-DFS y el mismo **consumo de memoria** que TREE-BFS, ambos exponenciales.

Sin embargo, **con una heurística apropiada, la cantidad de nodos generados puede reducirse drásticamente.**

Diferencia UCS y GBFS:

- UCS garantiza optimalidad, pero genera todos los nodos con costo de camino menor al de una solución óptima.
- GBFS puede reducir drásticamente la cantidad de nodos generados con una función heurística apropiada, pero no garantiza optimalidad.

Búsqueda A* o A star

Combina las estrategias **UCS** y **GBFS**.

Elige el nodo a expandir según una combinación de su **costo de camino** y su **valor heurístico**.

Se define una **función de evaluación** f tal que, dado un nodo n , $f(n) = n.\text{costo} + h(n)$

$f(n)$ es una **estimación del costo de una solución que pasa por el estado de n** .

El nodo n que se extrae de la frontera es siempre el de **menor valor de evaluación** $f(n)$

Implementación A*

- ¿Cómo elegimos de la frontera el próximo nodo a expandir? El nodo n con la menor evaluación $f(n)$.
- ¿Cómo lo logramos? Al igual que en UCS y GBFS, usando el TAD **cola de prioridad** para la frontera pero ordenando los nodos por la función de evaluación f .



Algoritmo A* de grafo

```
1 function GRAPH-ASTAR(problema) return solución o fallo
2   n0 ← NODO(problema.estado-inicial, None, None, 0)
3   frontera ← ColaPrioridad()
4   frontera.encolar(n0, n0.costo + problema.h(n0))
5   alcanzados ← {n0.estado: n0.costo}
6   do
7     if frontera.vacia() then return fallo
8     n ← frontera.desencolar()
9     if problema.test-objetivo(n.estado) then return solución(n)
10    forall a in problema.acciones(n.estado) do
11      s' ← problema.resultado(n.estado, a)
12      c' ← n.costo + problema.costo-individual(n.estado, a)
13      if s' is not in alcanzados or c' < alcanzados[s'] then
14        n' ← NODO(s', n, a, c')
15        alcanzados[s'] ← c'
16        frontera.encolar(n', n'.costo + problema.h(n'))
```

Suponemos que el problema tiene un **método** h que implementa la heurística.

Al igual que UCS, es importante que el test objetivo se llame al sacar de la frontera.

TREE-ASTAR es análogo. Hay que borrar únicamente las partes relacionadas con el diccionario de alcanzados.

La completitud y optimalidad dependen de la heurística.

Definición. Una heurística h es **admisible** si, para todo nodo n , $h(n)$ nunca sobrestima el mínimo costo de camino desde el estado de n a un estado objetivo

Definición. Una heurística h es **consistente** si, para todo nodo n e hijo n' generado por una acción a , se cumple $h(n) \leq \text{COSTO-INDIVIDUAL}(n.\text{estado}, a) + h(n')$.

Propiedad. Toda heurística consistente es admisible.

Teorema.

- **TREE-ASTAR** garantiza completitud y optimalidad si la heurística h es **admisible**.
- **GRAPH-ASTAR** garantiza completitud y optimalidad si la heurística h es **consistente**.

UNIDAD 3 PARTE 2 – INFORMADAS

Definición. Sean dos heurísticas h_1 y h_2 . Se dice que h_2 **domina** a h_1 si, para todo nodo n , $h_2(n) \geq h_1(n)$

Teorema. Sean dos heurísticas h_1 y h_2 tales que h_2 domina a h_1 , **A* con h_2 nunca expande más nodos que A* con h_1**

Problema relajado

Un **problema relajado** se obtiene quitando restricciones a las acciones de un problema.

Sea P' un problema relajado de un problema P .

1. El grafo de espacio de estados de P es un **subgrafo** del de P' .
2. El costo de una solución óptima en P' es una **cota inferior** del costo de una solución óptima en P .
3. Si P' **se resuelve sin búsqueda** (por ej. con una fórmula cerrada), podemos derivar una **heurística** para P .

Relajación N°1 $\rightarrow$ Heurística h_1 .

Relajación N°2 $\rightarrow$ Heurística h_2

Proposición. Sean P' un problema relajado de un problema P y h la heurística derivada de P' , entonces h es **consistente** (y por ende, también **admisible**) para P .

Subproblema

Un **subproblema** consiste en resolver una parte de un problema más grande, se obtiene quitando restricciones al objetivo de un problema.

Sea un subproblema P' de un problema P ,

1. El costo de una solución óptima de P' es una **cota inferior** del costo de una solución óptima de P .

 **Ejemplo.** Luego de resolver el subproblema 1-2-3-4, aún quedan por ubicar las demás fichas 5,6,7,8 (si es que no están ya en su posición correcta).

2. Si P' **se resuelve rápidamente con una búsqueda** (por ej. si su espacio de estados es mucho más chico), podemos derivar una **heurística** para P .

Una **base de dato de patrones** precalcula el costo de una solución óptima **para cualquier estado inicial de un subproblema** y lo almacena en memoria

Heurística compuesta

Sean $h_1, \dots, h_m$ heurísticas para un problema y tales que ninguna domina la otra.

Se define la **heurística compuesta** $h(n) = \max\{h_1(n), \dots, h_m(n)\}$

Proposiciones.

- Si $h_1, \dots, h_m$ son admisibles, entonces h es **admisible**.
- Si $h_1, \dots, h_m$ son consistentes, entonces h es **consistente**.
- h **domina** a $h_1, \dots, h_m$

UNIDAD 4 PARTE 1 – BUSQ ADVERSARIOS

Un **agente** es cualquier cosa capaz de percibir su **entorno** con la ayuda de **sensores** y actuar en ese medio utilizando **actuadores**.

En un entorno **multiagente**, el comportamiento de un agente depende del comportamiento de los demás. Es **competitivo** si los objetivos de los agentes están en conflicto. De lo contrario, se dice **cooperativo**.

Un problema de **búsqueda entre adversarios**, involucra un entorno multiagente y competitivo. Formalmente, se define por:

1. **S₀. Estado inicial.**
2. **JUGADOR(S).** Función que determina qué jugador tiene el turno en ese estado.
3. **ACCIONES(S).** Función que devuelve el conjunto de **acciones** legales en ese estado.
4. **RESULTADO(S,A).** Es el **modelo de transiciones**, que define el estado resultado de aplicar esa acción en ese estado.
5. **TEST-TERMINAL(S).** Función que determina si el juego está terminado en ese estado. Los estados en donde el juego ha finalizado se llaman **estados terminales**.
6. **UTILIDAD(S,P).** **Función de utilidad** que asigna un valor numérico (recompensa) a cada jugador P en un estado terminal S.  **Ejemplo.** En ajedrez la utilidad puede ser 1, 0 o ½ según si el jugador gana, pierde o empata.

Tipos de juegos

Nos limitaremos a juegos con:

- **Dos jugadores** que juegan por **turnos**. Los jugadores se llaman **MAX** y **MIN**, siendo MAX quién juega primero. Luego juega MIN, luego MAX de vuelta, y así sucesivamente.

- **Suma cero.** La suma de las utilidades de los jugadores es la misma para todo estado terminal.

 **Ejemplo.** El ajedrez es un juego de suma cero porque las utilidades en los estados terminales suman 1 (gana-pierde: 1+0, pierde-gana: 0+1, empatan: ½ + ½).

- **Información perfecta.** Al tomar una decisión, cada jugador está perfectamente informado de todos los eventos ocurridos previamente, incluida la inicialización. **Ejemplo.** El ta-te-ti tiene información perfecta. Por el contrario, el poker tiene información **imperfecta**, ya que no se ven las cartas de los otros jugadores.

Árbol de juego El estado inicial, las acciones y el modelo de transiciones definen un **árbol de juego**, donde los nodos representan estados y las ramas representan acciones, similar a un árbol de búsqueda.

Una **estrategia óptima** produce un resultado al menos tan bueno como cualquier otra estrategia cuando se juega contra un oponente **infalible** (es decir, que juega **óptimamente**).

Minimax Dado un árbol de juego, el **valor minimax** de un nodo es la **mejor utilidad** que puede alcanzar MAX cuando el juego se encuentra en ese estado, asumiendo que ambos jugadores juegan **óptimamente** desde allí hasta el final del juego.

Los valor minimax permiten hallar una estrategia óptima.

- **Estrategia minimax:**

- **MAX** elige la acción que lleva al hijo con **mayor valor minimax**.

- **MIN** elige la acción que lleva al hijo con **menor valor minimax**.

- **La estrategia minimax es una estrategia óptima** (asumiendo que ambos jugadores juegan óptimamente).

Performance de MINIMAX

- Hace una exploración **primero en profundidad** (recursiva) del árbol de juego.

- Sea un árbol de juego finito de altura **m** y factor de ramificación **b**.

- **Consumo de tiempo.** Genera a lo sumo b^{m+1} nodos.
- **Consumo de memoria.** Almacena a lo sumo $b * m$ llamadas recursivas

UNIDAD 4 PARTE 2 – BUSQ ADVERSARIOS

Juegos estocásticos En los **juegos estocásticos** suceden eventos externos impredecibles. Elementos **aleatorios** que combinan **suerte y habilidad**.

- Se incorporan **nodos de azar** para representar los posibles eventos aleatorios.
- Los nodos ya **no tienen valores minimax exactos**.
- Podemos calcular el **valor esperado** en un nodo de azar: **promedio ponderado** entre todos los posibles resultados.

Performance de minimax-esperado

- Minimax genera a lo sumo b^{m+1} nodos, siendo b el factor de ramificación y m es la altura del árbol anterior.
- Minimax-esperado considera **todos los posibles eventos aleatorios**. ► Minimax-esperado genera a lo sumo $(b * n)^{m+1}$, siendo n el número de eventos posibles.
- El mayor consumo de tiempo de minimax-esperado comparado al de minimax vuelve **poco realista considerar el árbol de juego completo**.
- En la práctica, se introducen un **test de corte** y una **función de evaluación**, al igual que en minimax.
- Incluso así, no es esperable que podamos mirar demasiado hacia adelante, **solo unos pocos niveles**.

Poda alfa-beta

Es posible computar la estrategia minimax **sin mirar todos** los nodos del árbol de juego.

Basta con **podar** aquellos nodos para los cuales se tenga evidencia suficiente de que **no influyen** en la decisión final. Esta técnica se llama **poda alfa-beta**

Si MAX tiene una opción n , pero más arriba en el árbol MIN tiene una opción m mejor (menor valor) que n , entonces n **nunca será alcanzado en una jugada real**.

Si se tiene suficiente información sobre m y n para llegar a esta conclusión, entonces es posible podar nodos

Supongamos que:

1. Computamos el valor minimax de m y es un número β . Es decir, MIN tiene una opción de valor β .
2. Más tarde, mientras computamos el valor minimax de n , logramos concluir que es mayor a β .

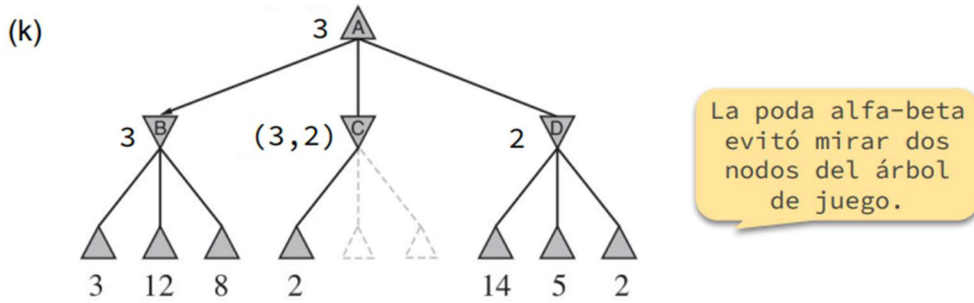
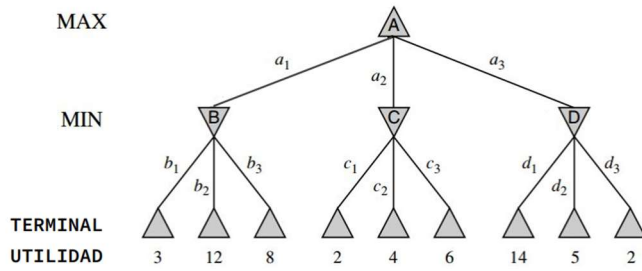
Entonces, podemos dejar de computar el valor minimax del padre de n **de inmediato**, es decir, **podando** los hijos que queden por calcular

Durante la ejecución, se mantienen dos parámetros (α, β) .

α : valor de la mejor decisión (mayor valor) encontrada hasta ahora en el camino para MAX.

β : valor de la mejor decisión (menor valor) encontrada hasta ahora en el camino para MIN.

Vamos a aplicar el algoritmo **MINIMAX** con poda alfa-beta sobre el siguiente árbol de juego.



Implementación de la poda alfa beta

- Las funciones MINIMAX-MAX y MINIMAX-MIN se aumentan con dos nuevos parámetros: α y β .
- MINIMAX-MAX:
 - Actualiza α si el valor minimax del nodo es más grande que α .
 - Termina la llamada recursiva si un hijo tiene valor minimax más grande que β .
- MINIMAX-MIN:
 - Actualiza β si el valor minimax del nodo es más chico que β .
 - Termina la llamada recursiva si un hijo tiene valor minimax más chico que α .

Performance de MINIMAX-ALFA-BETA

- Devuelve lo mismo que MINIMAX sin poda. Es decir, la estrategia óptima.
- Menor consumo de tiempo que MINIMAX sin poda, aunque sigue siendo exponencial en la altura.
- Es muy dependiente del orden en el que se examinan los sucesores